

Jonathan Vergonio
DSCI 551
Foundations of Data Management
08 May 2026

Final Report: Sales Analytics Dashboard via PostgreSQL

Introduction

This project explores how PostgreSQL can support an e-commerce sales analytics dashboard. The goal is to build a full-stack application that stores online retail transaction data, allows users to search and filter orders, and displays sales insights such as revenue trends, top products, and revenue by country.

PostgreSQL was chosen because it is a relational database system with strong support for structured data, B-tree indexing, query planning, aggregation, and 'EXPLAIN ANALYZE'. These features make it useful for analyzing how database internals affect application performance. The application addresses the problem of turning raw transaction data into useful business insights while also showing how PostgreSQL executes queries behind the scenes.

Database System Overview and Architecture

PostgreSQL is an open-source relational database system used to store, manage, and query structured data. It is commonly used for applications that need reliable transactions, strong data consistency, and support for complex SQL queries. In this project, PostgreSQL stores the e-commerce transaction data and supports both user-facing queries (product search/country filtering) and analytical dashboard features (providing KPIs and revenue charts).

The database system was a good choice for this project because it supports structured tables, SQL aggregation functions, indexing, and query performance analysis. Indexing helps PostgreSQL find rows faster without scanning the entire 'orders' table every time. The dashboard uses PostgreSQL to store order records, retrieve filtered results, calculate revenue metrics, and display query execution behavior using 'EXPLAIN ANALYZE'. These features make PostgreSQL useful for business analytics applications where users need both accurate transactional data and fast analytical queries.

PostgreSQL is commonly used for web applications, financial systems, analytics dashboards, inventory systems, and applications that require strong data integrity. For this project, its relational model fits the Online Retail dataset well because each transaction has structured fields such as invoice number, product, quantity, order date, unit price, customer ID, and country.

Internal Architecture

PostgreSQL uses an internal architecture based on storage, indexing, query planning, query execution, and transaction management. Data is stored in heap files, meaning rows are not

physically sorted by a specific column. Because of this, indexes are important for helping PostgreSQL find relevant rows more efficiently.

A major focus of this project is B-tree indexing. B-tree indexes are PostgreSQL's default index type and are useful for exact-match and range queries. In this project, indexes on `order_date` and `product` support features such as recent-order retrieval and product search. However, PostgreSQL does not always use an index; the cost-based query planner decides whether an index scan or sequential scan is more efficient based on table size, available indexes, and estimated matching rows.

After the planner chooses a strategy, the query executor runs the query. In this application, 'EXPLAIN ANALYZE' is used to show the actual execution plan, including scan type, sorting, aggregation, planning time, and execution time. This allows the dashboard to connect user-facing features, such as recent orders, KPI cards, and revenue charts, to PostgreSQL's internal execution behavior.

PostgreSQL also supports Multi-Version Concurrency Control, or MVCC, which allows reads and writes to happen at the same time by maintaining different row versions. This relates to the insert-order feature because new orders can be added while users continue querying the dashboard. Distribution and replication were not implemented because the project runs locally on a single PostgreSQL database, although PostgreSQL does support replication for larger production systems.

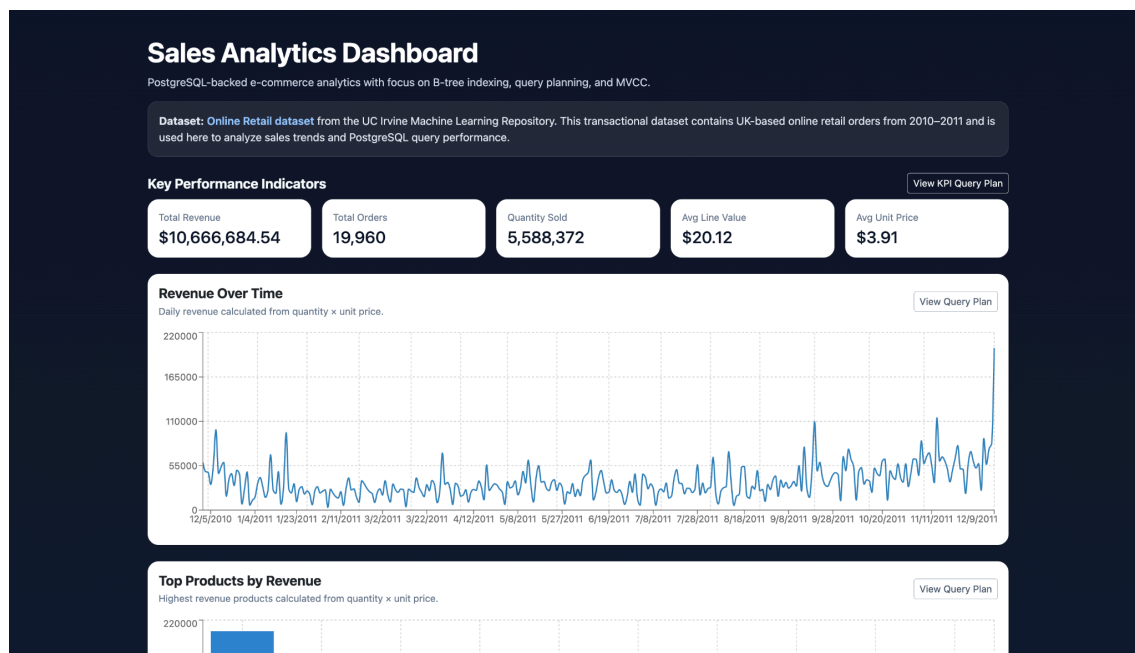


Figure 1. Dashboard overview showing the dataset source, KPI cards, and revenue-over-time visualization.

Application Design

The application is an e-commerce sales analytics dashboard that allows users to explore and analyze transaction data stored in PostgreSQL. It uses the Online Retail dataset from the UC Irvine Machine Learning Repository, which contains sales transactions from a UK-based online retailer. The data is loaded into a PostgreSQL orders table with fields such as invoice_no, stock_code, product, quantity, order_date, unit_price, customer_id, and country.

The application is built as a full-stack web application. The frontend is developed with React and displays KPI cards, charts, search/filter controls, an insert form, an orders table, and query plan outputs. The backend is built with Node.js and Express, which provides REST API routes for retrieving recent orders, searching products, filtering by country, inserting new orders, and returning PostgreSQL 'EXPLAIN ANALYZE' results. PostgreSQL serves as the database layer where the transaction data is stored, indexed, queried, and aggregated.



Figure 2. Comparative Bar and Pie Charts utilizing Retail data, illustrating ‘Top Products by Revenue’ and ‘Revenue by Country’ analysis with an Include/Exclude UK toggle.

Users interact with the dashboard by running queries such as viewing recent orders, searching for products, filtering by country, and inserting a new order. The dashboard also includes analytical features such as total revenue, total orders, revenue over time, top products by revenue, and revenue by country. For several of these features, users can view the PostgreSQL query plan to understand how the database executes the query, including scan type, sorting, aggregation, planning time, and execution time.

The overall workflow is: the user interacts with the React dashboard, the frontend sends a request to the Express backend, the backend runs a SQL query against PostgreSQL, and the results are returned to the frontend for display as tables, KPI cards, charts, or query plans.

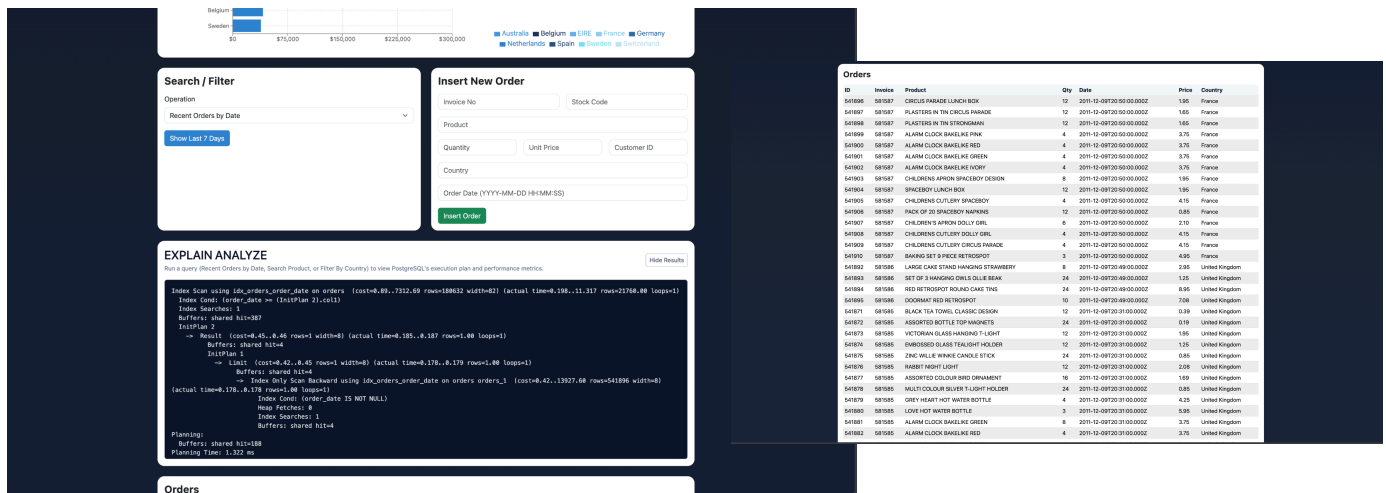


Figure 3. PostgreSQL ‘EXPLAIN ANALYZE’ output after running the “Recent Orders by Date” query. The output shows query execution behavior, including index usage, planning time, and execution time.

Mappings Between DB Internals and Application Behaviors

1. Querying Recent Orders by Date

Application Behavior: The user can select “Recent Orders by Date” to view the most recent transactions in the dataset. Since the dataset is historical, the query uses the maximum ‘order_date’ in the table and retrieves records from the previous seven days instead of using the current system date.

```
SELECT id, invoice_no, product, quantity, order_date, unit_price, country
FROM orders
WHERE order_date >= (
    SELECT MAX(order_date) - INTERVAL '7 days'
    FROM orders )
ORDER BY order_date DESC
LIMIT 50;
```

```

EXPLAIN ANALYZE
Run a query (Recent Orders by Date, Search Product, or Filter By Country) to view PostgreSQL's execution plan and performance metrics.
Hide Results

Index Scan using idx_orders_order_date on orders (cost=0.09..7312.69 rows=180632 width=82) (actual time=0.042..12.328 rows=21760.00 loops=1)
  Index Cond: (order_date >= (InitPlan 2).coll)
  Index Searches: 1
  Buffers: shared hit=387
  InitPlan 2
    -> Result (cost=0.45..0.46 rows=1 width=8) (actual time=0.030..0.032 rows=1.00 loops=1)
      Buffers: shared hit=4
      InitPlan 1
        -> Limit (cost=0.42..0.45 rows=1 width=8) (actual time=0.026..0.027 rows=1.00 loops=1)
          Buffers: shared hit=4
          -> Index Only Scan Backward using idx_orders_order_date on orders orders_1 (cost=0.42..13927.60 rows=541896 width=8)
            (actual time=0.025..0.026 rows=1.00 loops=1)
            Index Cond: (order_date IS NOT NULL)
            Heap Fetches: 0
            Index Searches: 1
            Buffers: shared hit=4
      Planning Time: 0.171 ms
      Execution Time: 14.000 ms

```

Result: The query plan shows an Index Scan using `idx_orders_order_date`, meaning PostgreSQL used the B-tree index on ‘order_date’ instead of scanning the entire table. This improves the recent-orders feature because the dashboard can retrieve time-based records more efficiently. The planning and execution times shown in ‘EXPLAIN ANALYZE’ provide evidence of the query’s performance.

2. Product Search

Application Behavior: The user can search for orders by product name. For example, searching for “WHITE HANGING HEART” returns matching products in the orders table.

```

SELECT id, invoice_no, product, quantity, order_date, unit_price, country
FROM orders
WHERE product ILIKE '%WHITE HANGING HEART%'
ORDER BY order_date
LIMIT 50;

```

Database Internals Mapping: The project creates a B-tree index on product, but this query uses ILIKE with a leading wildcard. Because the search pattern is ‘%term%’, PostgreSQL may not be able to use a normal B-tree index efficiently. Instead, the query plan may show a sequential scan or parallel sequential scan because PostgreSQL must check many rows for a partial text match.

```

EXPLAIN ANALYZE
Run a query (Recent Orders by Date, Search Product, or Filter By Country) to view PostgreSQL's execution plan and performance metrics.
Hide Results

Gather (cost=1000.00..12050.27 rows=2609 width=82) (actual time=0.578..0.579 rows=2369.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=7976
  -> Parallel Seq Scan on orders (cost=0.00..10798.38 rows=1087 width=82) (actual time=0.237..0.650 rows=789.67 loops=3)
    Filter: (product ~* 'WHITE HANGING HEART'::text)
    Rows Removed by Filter: 179847
    Buffers: shared hit=7976
  Planning:
    Buffers: shared hit=140
  Planning Time: 6.179 ms
  Execution Time: 85.969 ms

```

Result: The ‘EXPLAIN ANALYZE’ output shows a Parallel Seq Scan, meaning PostgreSQL split the table scan across multiple workers instead of using the B-tree index on ‘product’. This happens because the query uses ILIKE ‘%WHITE HANGING HEART%’, and the leading wildcard makes a normal B-tree index less useful. PostgreSQL still filters the rows correctly, but

it must check many rows for the substring match, which makes this query slower than the indexed recent-orders query.

3. Country Filter as a Non-Indexed Query

Application Behavior: The user can filter orders by country, such as France. The dashboard sends the selected country to the backend and displays matching orders.

```
SELECT id, invoice_no, product, quantity, order_date, unit_price, country
FROM orders
WHERE country = 'France'
ORDER BY order_date
LIMIT 50;
```

Database Internals Mapping: In the current database setup, indexes are created on `order_date` and `product`, but not on `country`. Because there is no B-tree index on 'country', PostgreSQL must evaluate rows in the heap table to find matches. In the `EXPLAIN ANALYZE` output, this can appear as a sequential scan or parallel sequential scan.



Result: The 'EXPLAIN ANALYZE' output shows a Parallel Seq Scan, meaning PostgreSQL scanned the orders table using multiple workers instead of using an index. This happens because the current database has indexes on `order_date` and `product`, but not on `country`. As a result, PostgreSQL must check many rows to find orders where `country = 'France'`, which shows the performance cost of filtering on a non-indexed column.

Additional Mappings to Internal Behavior:

4. KPI Cards and Aggregation: The KPI cards summarize the dataset by calculating total revenue, total orders, quantity sold, average line value, and average unit price. This maps to PostgreSQL aggregation because the database uses functions such as `COUNT`, `SUM`, and `AVG` to compute summary metrics from many rows. The 'EXPLAIN ANALYZE' output shows that PostgreSQL scans and aggregates a large portion of the table, which is expected because these metrics require dataset-wide calculations.

5. Revenue Over Time: The revenue-over-time chart groups orders by `DATE(order_date)` and calculates daily revenue using `SUM(quantity * unit_price)`. This maps to grouping, aggregation, and sorting inside PostgreSQL. The query plan shows operations such as aggregation and sort because PostgreSQL must group many transactions into daily totals before sending the result to

the chart.

6. Top Products by Revenue: The top products chart groups orders by product, calculates revenue for each product, sorts by revenue, and returns the top 10. This maps to PostgreSQL aggregation, sorting, and limiting. Instead of React calculating revenue from raw rows, PostgreSQL performs the heavier SQL work and returns a small result for visualization.

7. Revenue by Country: The revenue-by-country chart groups orders by country and calculates total revenue for each country. This maps to aggregation and sorting, and since ‘country’ is not indexed in the current setup, PostgreSQL may scan the table before grouping results. The include/exclude UK toggle is a frontend design choice because the UK dominates the dataset and makes smaller country differences harder to compare.

8. Insert New Order: The insert form allows users to add a new order to the PostgreSQL orders table. This maps to PostgreSQL’s transaction system and MVCC because new rows can be inserted while read queries continue to return consistent results. The app does not use ‘EXPLAIN ANALYZE’ for inserts because it would actually execute the insert, but the feature still demonstrates transactional updates alongside analytics.

Comparison with MySQL and MongoDB

PostgreSQL, MySQL, and MongoDB can all store application data, but they are designed around different data models and strengths. PostgreSQL and MySQL are both relational database systems, meaning they organize data into tables with rows, columns, schemas, and SQL queries. This makes both strong choices for structured data like the Online Retail dataset, where each order has consistent fields such as invoice number, product, quantity, date, price, and country.

MySQL supports many of the same basic SQL operations used in this project, including filtering, aggregation, indexing, and query explanation. However, PostgreSQL was a better fit because it provides stronger query-plan visibility and a wider set of advanced database features, such as partial indexes, expression indexes, materialized views, and extensions. These features make PostgreSQL especially useful for analytical applications and projects focused on understanding database internals.

MongoDB is different because it is a document-oriented NoSQL database instead of a relational database. Rather than storing data in tables, MongoDB stores data as flexible JSON-like documents. This is useful when the data structure changes often or contains nested fields, but it is less natural for this project because the Online Retail dataset is already highly structured. MongoDB can still perform filtering and aggregation using its aggregation pipeline, but SQL databases like PostgreSQL and MySQL are usually more straightforward for grouped revenue calculations and structured analytics.

Overall, PostgreSQL was the best fit for this project because it combines reliable transactional storage, SQL-based analytics, indexing, and detailed query-plan analysis. MySQL would be a good alternative for traditional relational web applications, while MongoDB would be better for flexible document-based applications.

Limitations and Lessons Learned

One limitation of the project is that it runs locally rather than being deployed online, so the database, backend, and frontend must be configured on the user’s machine. The dashboard also focuses on core sales analytics, so deeper analysis like customer segmentation, seasonality, forecasting, and product-category trends could be added in the future.

A major challenge was making sure the Online Retail dataset was loaded correctly into PostgreSQL. The original dataset column names had to be matched to the application's schema, and the data needed to be cleaned into usable fields such as invoice_no, product, quantity, order_date, unit_price, and country. This showed how important the data ingestion step is, because incorrect column names or data types can affect the entire application.

Through this project, I learned how PostgreSQL indexing and query planning affect application performance. I was able to compare sequential scans, parallel sequential scans, and index scans using 'EXPLAIN ANALYZE'. I also learned that indexes do not always guarantee faster queries because PostgreSQL's planner chooses the execution strategy based on estimated cost. In general, an index scan may have slightly more planning overhead, but it can reduce execution time when the query is selective. However, for queries that scan a large portion of the table, PostgreSQL may choose a sequential or parallel sequential scan because that can be more efficient.

Future improvements would include adding more detailed analysis of the retail dataset, such as customer-level patterns, monthly revenue trends, and product category analysis. The dashboard could also include more complex visualizations and an improved date-range filter so users can choose custom time periods instead of only viewing recent orders. Another improvement would be to parse the 'EXPLAIN ANALYZE' output into simpler labels, such as scan type, index used, planning time, and execution time, so users can understand query performance more easily.

Project Links

GitHub Repository Link: <https://github.com/jonvrg/sales-analytics-postgresql/tree/main>

Link to Dataset: <https://archive.ics.uci.edu/dataset/352/online+retail>

References

GeeksforGeeks. "What is MySQL?"

<https://www.geeksforgeeks.org/mysql/what-is-mysql/>

GeeksforGeeks. "What is MongoDB? Working and Features."

<https://www.geeksforgeeks.org/mongodb/what-is-mongodb-working-and-features/>

PostgreSQL Global Development Group. "Multi-Version Concurrency Control (MVCC) Introduction." *PostgreSQL Documentation*.

<https://www.postgresql.org/docs/current/mvcc-intro.html>

PostgreSQL Global Development Group. "B-Tree Indexes." *PostgreSQL Documentation*.

<https://www.postgresql.org/docs/current/btree.html>